Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 3

Aim:

To apply Decision Tree and Random Forest for classification tasks.

Theory:

Dataset Link: Mushroom Classification

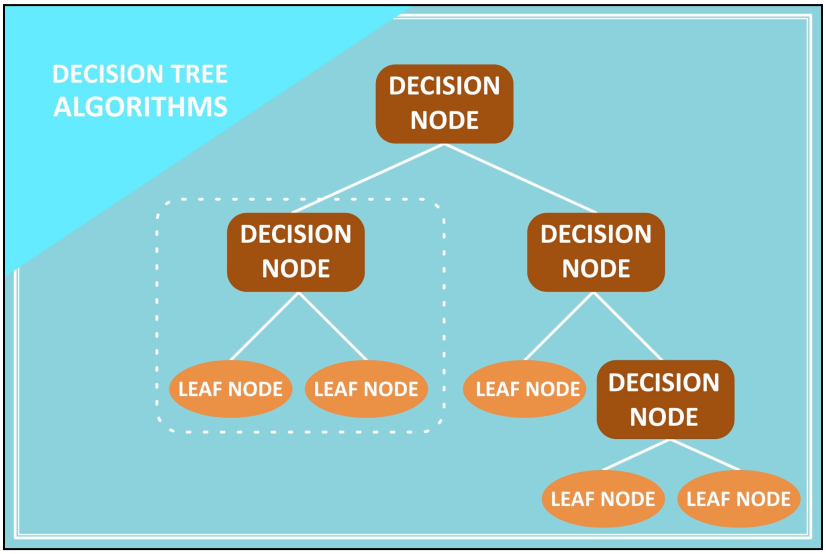
Dataset Description:
This dataset includes descriptions of hypothetical samples corresponding to 23 species of gilled mushrooms in the Agaricus and Lepiota Family. Each species is identified as definitely edible, definitely poisonous, or of unknown edibility and not recommended. This latter class was combined with the poisonous one.

- Objective: To classify a mushroom as Edible (e) or Poisonous (p) based on physical characteristics.
- Records: 8124 instances.
- Features: 22 categorical predictors including cap-shape, cap-color, gill-color, odor, etc.
- Target: class (e=edible, p=poisonous).

Variable	Description	Data Type	Values (Encoded Example)
class	Target Label	Categorical	p (poisonous), e (edible)
cap-shape	Shape of the cap	Categorical	bell, conical, convex, flat, knobbed, sunken
cap-surface	Texture of cap	Categorical	fibrous, grooves, scaly, smooth
cap-color	Color of cap	Categorical	brown, buff, cinnamon, gray, green, etc.
odor	Smell of mushroom	Categorical	almond, anise, creosote, fishy, foul, etc.
gill-color	Color of gills	Categorical	black, brown, buff, chocolate, gray, etc.
habitat	Where it grows	Categorical	grasses, leaves, meadows, paths, urban, waste, woods

Decision Tree

A Decision Tree is a flowchart-like tree structure where an internal node represents a feature (or attribute), the branch represents a decision rule, and each leaf node represents the outcome. The topmost node in a decision tree is known as the root node. It partitions the tree in a recursive manner called recursive partitioning.



Mathematical Formula:

The core algorithm used is ID3 or CART. The decision of where to split the data is based on metrics like Entropy and Information Gain or Gini Impurity.

Entropy (H): Measures the impurity or randomness in the data.

$$H(S) = -\sum p_i \log_2(p_i)$$

Where pi is the probability of class i.

Information Gain (IG): Measures the reduction in entropy after splitting on a specific attribute A.

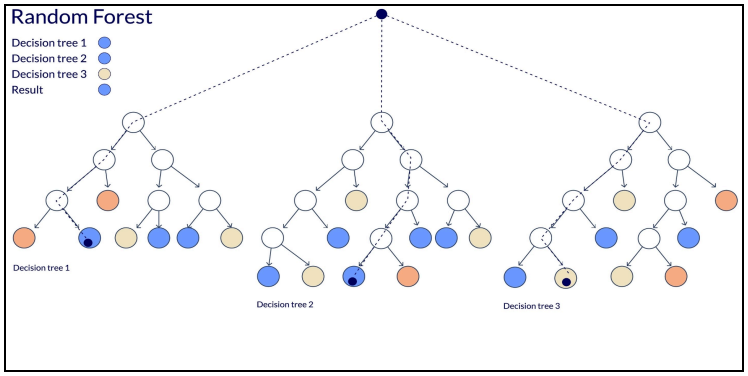
$$IG(S, A) = H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$$

Gini Impurity: (Used by CART, default in sklearn)

$$Gini = 1 - \sum (p_i)^2$$

Random Forest Classifier

Random Forest is an ensemble learning method that operates by constructing a multitude of decision trees at training time. For classification tasks, the output of the random forest is the class selected by most trees (mode). It corrects for decision trees' habit of overfitting to their training set.

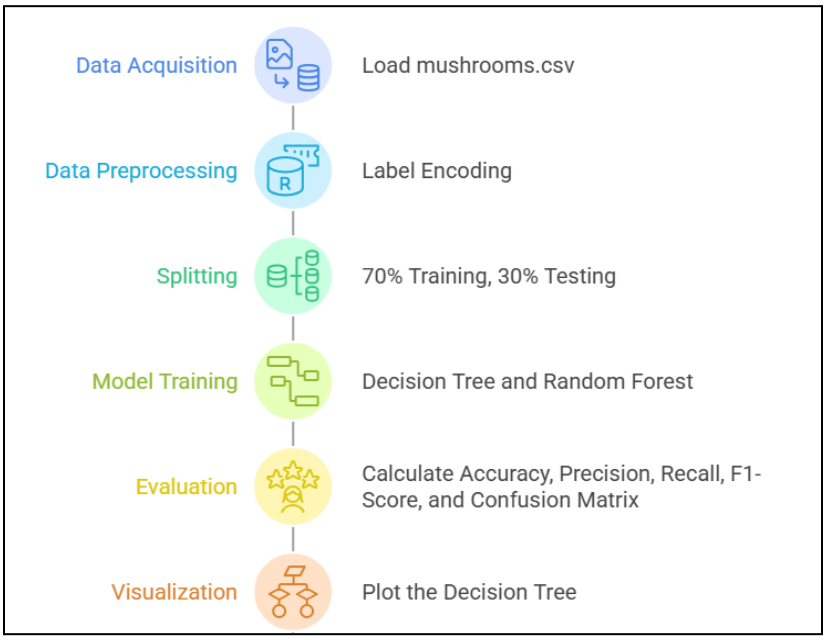


Working Principle: (Bagging)

- **Bootstrapping:** Random samples are selected from the dataset with replacement.
- **Parallel Training:** Multiple decision trees are trained on these different samples.
- **Feature Randomness:** At each split in a tree, only a random subset of features is considered.
- **Aggregation:** The final prediction is the majority vote of all individual trees.

Workflow:

- **Data Acquisition:** Load mushrooms.csv.
- **Data Preprocessing:**
  - The dataset is entirely categorical (text). Machine learning models require numbers.
  - **Label Encoding:** Convert all columns (e.g., 'bell', 'conical') into integers (0, 1, 2...).
- **Splitting:** Partition data into 70% Training and 30% Testing sets.
- **Model Training:**
  - **Decision Tree:** Train using DecisionTreeClassifier with gini criterion.
  - **Random Forest:** Train using RandomForestClassifier with 100 trees (n_estimators=100).
- **Evaluation:** Calculate Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- **Visualization:** Plot the Decision Tree to visualize the rules.



Performance Analysis:

Decision Tree Performance

- **Accuracy:** 100%
- **Analysis:** The Decision Tree achieved perfect classification. This is common with the Mushroom dataset because the feature **"odor"** is a near-perfect predictor (e.g., foul smell = always poisonous).
- **Risk:** While 100% looks good, a single tree can be prone to **overfitting**, meaning it memorizes the training data rules perfectly but might fail on slightly different future data.

Random Forest Performance

- **Accuracy:** 100%
- **Analysis:** The Random Forest also achieved 100%.
- **Advantage:** Even though both achieved 100%, the Random Forest is theoretically more **robust**. By averaging 100 different trees, it ensures that the model isn't relying on just one specific path of logic, making it safer for generalization.

Feature Importance

The models identified Odor as the most critical feature, followed by Gill Color and Spore Print Color. This aligns with mycological theory where smell is a primary identifier for toxicity.

Hyperparameter Tuning

We used GridSearchCV to optimize the Decision Tree to prevent it from becoming too complex (overfitting).

Hyperparameters Tuned:

- max_depth: Controls how deep the tree can grow. Deeper trees capture more details but can overfit.
- criterion: 'gini' vs 'entropy' (measures of impurity).

Best Parameters Found: {'criterion': 'gini', 'max_depth': None}

Algorithm Limitations

Decision Tree:

- **Overfitting:** Trees can create overly complex rules that do not generalize well.
- **Instability:** A small change in the data can result in a completely different tree being generated.

Random Forest:

- **Complexity:** Much harder to interpret than a single Decision Tree (Black Box model).
- **Computational Cost:** Requires more memory and time to train since it builds hundreds of trees.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
```

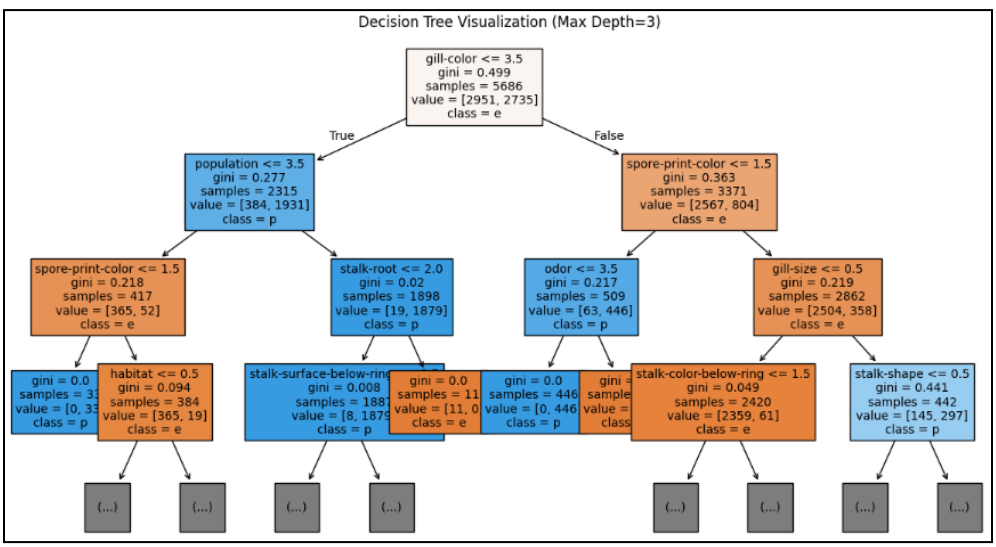
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder

# ═══════════════════════════════════════════
# 1. LOAD AND PREPROCESS DATA
# ═══════════════════════════════════════════

# 1. Load Dataset
try:
    df = pd.read_csv('Exp3-Mushrooms.csv')
    print("Dataset loaded successfully.")
except:
    print("Error: Please upload 'mushrooms.csv' to Colab.")

# 2. Encoding Categorical Variables
# The entire dataset is text strings. We need to convert them to numbers.
label_encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Display the mapping for the Target Class
print("\nClass Mapping:")
classes = label_encoders['class'].classes_
for i, item in enumerate(classes):
    print(f"{item} → {i}")
# Usually: e (edible) → 0, p (poisonous) → 1

# 3. Feature Selection
X = df.drop('class', axis=1)
y = df['class']

# 4. Split Data (70% Train, 30% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# ═══════════════════════════════════════════
# 2. DECISION TREE (PART A)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART A: DECISION TREE CLASSIFIER")
print("="*50)

# Train Model
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

# Evaluation
acc_dt = accuracy_score(y_test, y_pred_dt)
print(f"Decision Tree Accuracy: {acc_dt*100:.2f}%")
print("\nClassification Report (Decision Tree):")
print(classification_report(y_test, y_pred_dt))

# Visualization of the Tree (Truncated for visibility)
plt.figure(figsize=(15, 8))
plot_tree(dt_model, filled=True, feature_names=X.columns, class_names=classes, max_depth=3,
fontsize=10)
```

```
plt.title("Decision Tree Visualization (Max Depth=3)")
plt.show()

# ═══════════════════════════════════════════
# 3. RANDOM FOREST (PART B)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART B: RANDOM FOREST CLASSIFIER")
print("="*50)

# Train Model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

# Evaluation
acc_rf = accuracy_score(y_test, y_pred_rf)
print(f"Random Forest Accuracy: {acc_rf*100:.2f}%")
print("\nClassification Report (Random Forest):")
print(classification_report(y_test, y_pred_rf))

# ═══════════════════════════════════════════
# 4. HYPERPARAMETER TUNING (Decision Tree)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART C: HYPERPARAMETER TUNING (DT)")
print("="*50)

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5)
grid_search.fit(X_train, y_train)

print("Best Hyperparameters:", grid_search.best_params_)
print(f"Best Estimator Accuracy: {grid_search.best_score_*100:.2f}%")

# ═══════════════════════════════════════════
# 5. COMPARISON & VISUALIZATION
# ═══════════════════════════════════════════

# Confusion Matrices Side-by-Side
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap='Blues',
ax=axes[0])
axes[0].set_title('Decision Tree Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, fmt='d', cmap='Greens',
ax=axes[1])
axes[1].set_title('Random Forest Confusion Matrix')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
```

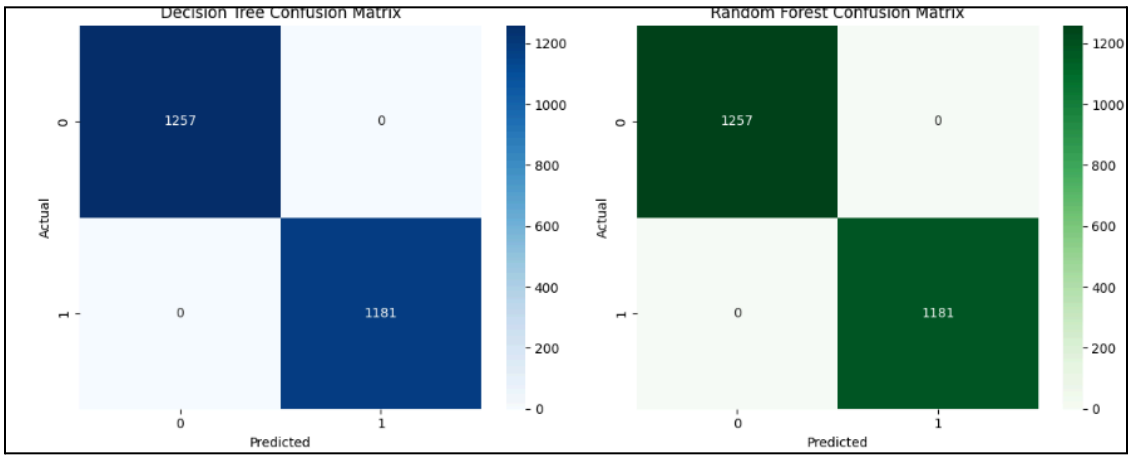
```
plt.tight_layout()
plt.show()

# Feature Importance (Random Forest)
importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1]
top_n = 10 # Show top 10 features

plt.figure(figsize=(10, 6))
plt.title("Top 10 Feature Importances (Random Forest)")
plt.bar(range(top_n), importances[indices][:top_n], align="center", color='orange')
plt.xticks(range(top_n), X.columns[indices][:top_n], rotation=45)
plt.tight_layout()
plt.show()
```

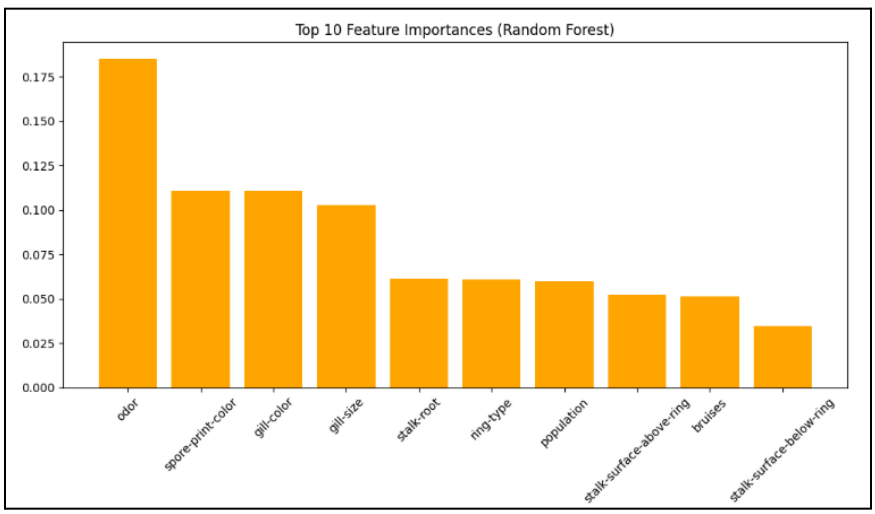
Output:

```
Class Mapping:
e -> 0
p -> 1

==================================================
PART A: DECISION TREE CLASSIFIER
==================================================
Decision Tree Accuracy: 100.00%

Classification Report (Decision Tree):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1257
           1       1.00      1.00      1.00      1181

    accuracy                           1.00      2438
   macro avg       1.00      1.00      1.00      2438
weighted avg       1.00      1.00      1.00      2438
```



```
==================================================
PART B: RANDOM FOREST CLASSIFIER
==================================================
Random Forest Accuracy: 100.00%

Classification Report (Random Forest):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1257
           1       1.00      1.00      1.00      1181

    accuracy                           1.00      2438
   macro avg       1.00      1.00      1.00      2438
weighted avg       1.00      1.00      1.00      2438


==================================================
PART C: HYPERPARAMETER TUNING (DT)
==================================================
Best Hyperparameters: {'criterion': 'gini', 'max_depth': None, 'min_samples_split': 2}
Best Estimator Accuracy: 100.00%
```





Conclusion:

In this experiment, we successfully implemented and compared Decision Tree and Random Forest classifiers on the Mushroom Classification dataset to predict edibility.

1. **Performance:** Both models achieved 100% accuracy, accurately classifying every mushroom in the test set as either edible or poisonous. This unusually high performance is attributed to the distinct, separable features in the dataset, particularly Odor, which serves as a near-perfect indicator of toxicity.

2. **Model Comparison:**
    - The **Decision Tree** provided a clear, interpretable set of rules (e.g., "If Odor is Foul, then Poisonous"), making it easy to understand the logic behind predictions.
    - The **Random Forest**, while less interpretable due to its ensemble nature, is theoretically more robust against overfitting and would likely perform better on noisier, less distinct datasets.
3. The analysis confirmed that **Odor**, **Gill Color**, and **Spore Print Color** are the most critical biological features for distinguishing poisonous mushrooms. The experiment demonstrates that for datasets with clear decision boundaries, simple models like Decision Trees can perform just as well as complex ensembles like Random Forests.